Shared Discovery for CollectionSpace and ArchivesSpace

Technical Specification

Unified Search and Display Interface for archives and material culture

Document Status: FINAL
Version: 0.5
Date: August 2026
Source Story: C3
Project: LYRASIS Interoperability Project
Systems: Shared Discovery app (UI + BFF) – ArchivesSpace Search API / PUI – CollectionSpace public gateway / Public Browser

Purpose and Scope
Background
System Overview
Actors and Deployment Context
Configuration Requirements
New components
Search backends
Intermediate Result Card Schema
Display Implementation Options
BFF search contract (sketch)
High-Level Sequence
API Dependencies
Behavior Scenarios
Error Scenarios
Other Requirements
Implementation Gaps

Purpose and Scope

This specification defines requirements for a unified search and display interface that surfaces records from both ArchivesSpace and CollectionSpace in a single search experience. Architecture is **federated live search**: a new shared-discovery application queries each system's public search backend in parallel, normalizes hits into result cards, and links out to the originating public UIs.

Assumptions

ID	Assumption
A-01	Implementation is **federated live search** via a new shared-discovery application (UI + BFF).
A-02	Scope is **one configured pair**: one ArchivesSpace instance (one or more repositories, configurable) + one CollectionSpace tenant.
A-03	Deliverable is a **new** shared-discovery application (public UI + BFF), not a plugin inside AS PUI or CS Public Browser.
A-04	**Click-through only:** result titles / "View full record" open the AS PUI or CS Public Browser in a **new tab**. No intermediate detail page in the shared UI.
A-05	AS record types in v1: `archival_object` and `digital_object`. Resources, agents, accessions, etc. deferred.
A-06	CS record types in v1: **collection objects** already exposed to the public browser / gateway (same publish gate as C1: Publish To `all` / `cspacepub`).
A-07	CS queries go through the **public gateway** → **Elasticsearch** path (anonymous), not authenticated Common Services staff search.
A-08	AS queries use the **backend Search API** (`/repositories/:id/search` or `/search`). Prefer a **read-only API user + session token** held only in the BFF. Backend port **8089 must not be exposed to browsers** — only to the BFF (allowlist). ArchivesSpace provides a Solr filter (`types:pui_record, publish:true`) that can be used in the request to ensure that only published records are returned; investigate something similar for CollectionSpace or determine that all CS results are public.

ID	Assumption
A-09	Results are presented **grouped by source** (see Display options). No global merged relevance ranking in v1.
A-10	Facets in v1 are **coarse**: source system; optional “has media/thumbnail”; optional simple date bounds if both adapters can supply them cheaply.
A-11	“Is it digitized / can I view the item?” is a **nice-to-have** signal on the card when detectable — not a v1 blocker.
A-12	Duplicate / related-record merging across AS and CS is **out of scope**.
A-13	**C1 is not required**.

Records in Scope

System	Records Searched and Displayed
ArchivesSpace	Archival Object records, Digital Object records
CollectionSpace	Object records, associated media

Out of scope: OAI-PMH harvesting and institutional discovery-platform configuration (Blacklight, VuFind, Primo, etc.), authentication/authorization for end users of the shared UI, and any changes to ArchivesSpace or CollectionSpace data models.

Background

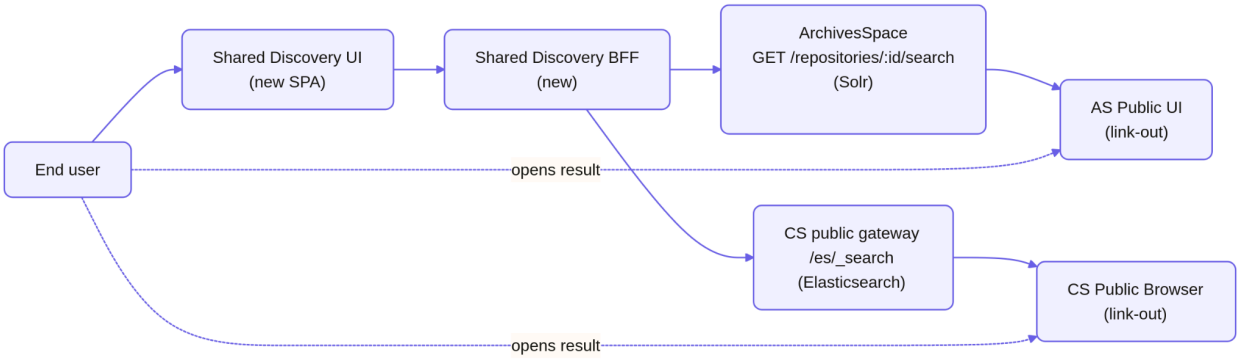
ArchivesSpace is an archives content management system with resource records (collections), archival object components (series, folders, items), and related digital objects and agents. CollectionSpace manages material culture records: individual objects, groups of objects (exhibitions), and associated media. Each system already exposes public search (ArchivesSpace Solr search API; CollectionSpace public gateway → Elasticsearch) and a public record UI (AS PUI; CS Public Browser).

Institutions with both ArchivesSpace and CollectionSpace sometimes hold historically related material across both systems. A researcher looking for records about a person, event, or topic may find relevant archival finding aids in ArchivesSpace and related museum objects in

CollectionSpace. A unified search interface that presents both — while clearly distinguishing them — significantly improves research usability.

System Overview

Architecture: Federated Live Search



On each search request the BFF (backend for frontend):

1. Issues **parallel** queries to AS and CS (subject to source filter).
2. Maps each hit into a shared **result card** schema.
3. Returns **grouped** result sets (one group per source) plus coarse filter metadata.
4. Never stores a full metadata mirror; optional short-lived response cache only.

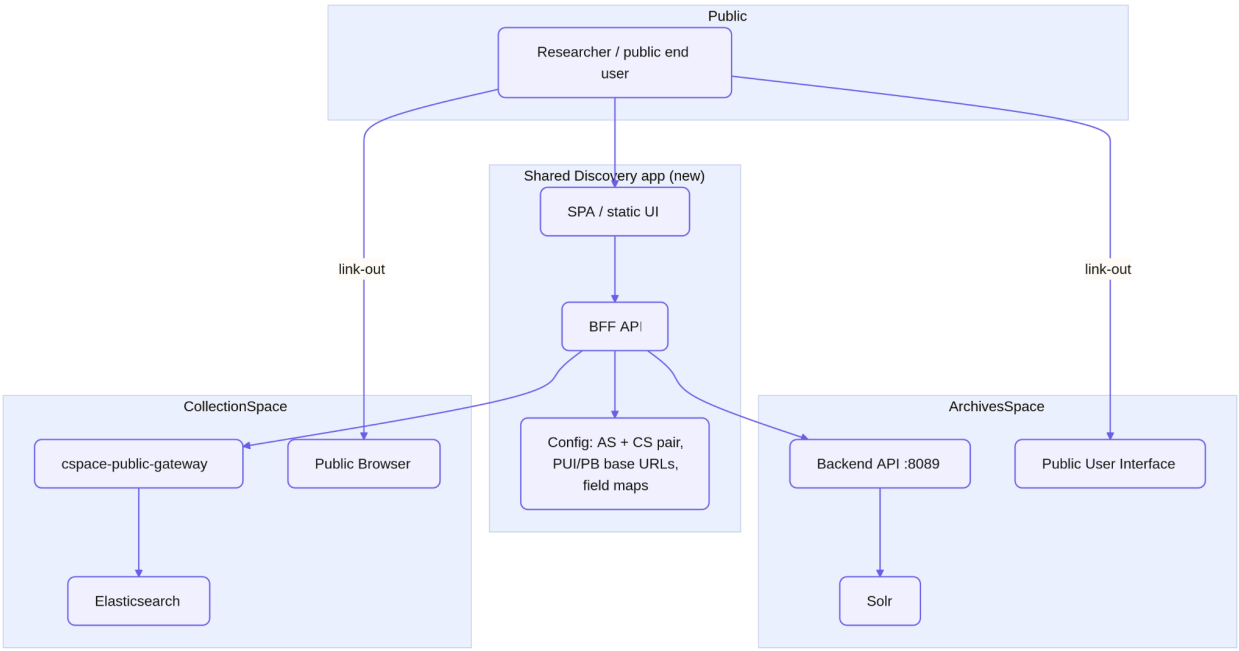
Pros: No harvest lag; reuses published search indexes;
no dependency on C1; smallest honest v1.
Cons: Relevance scores are **not comparable** across systems; facet vocabularies differ; one source down ⇒ partial results.

Only select fields are displayed to the user (see Display Implementation Options). The user will have a single search bar where they can key an initial search by keyword. Additional search refinement happens after the search is complete using filters and facets.

The keyword search covers the fields each adapter maps into the result card (and underlying IR search indexes), whether or not every field is shown in the preview display.

The user can follow links to the records in the source system (ArchivesSpace Public User Interface or CollectionSpace Public Browser), which includes additional metadata beyond the preview card.

Actors and Deployment Context



Actor	Role
End user	Keywords search; filters by source / coarse facets; opens source records
Shared Discovery Administrator	Configures the AS↔CS pair, base URLs, repository IDs, display labels, timeouts
ArchivesSpace Administrator	Ensures PUI URLs resolve; provides read-only API credentials (or confirms anonymous search policy)
CollectionSpace Administrator	Ensures public gateway + ES indexing for published collection objects; provides public browser base URL

Configuration Requirements

Item	Description
`as.baseApiUrl`	ArchivesSpace backend base (BFF-only network path)
`as.username` / `as.password`	Read-only API user (or equiv.)
`as.repositoryIds`	One or more repo IDs to search
`as.puiBaseUrl`	For link-out: `AppConfig[:public_proxy_url]` equivalent
`as.types`	Default `archival_object,digital_object`
`cs.gatewayUrl`	e.g. `https://cs.example.edu/gateway/museum`
`cs.publicBrowserBaseUrl`	Link-out base for object pages
`cs.esIndex` / query template	Match public browser expectations for the tenant/profile
`ui.defaultLayout`	`grouped` (A) or `unified` (B)
`ui.labels`	“Archival Records” / “Museum Objects” (see Gap G-05)
Timeouts / page size defaults	Fail a group fast rather than block the whole response

New components

Repositories / packages (proposed)

Component	Responsibility
`shared-discovery-ui`	Public SPA: search box, Option A/B layouts, coarse filters, pagination per group, link-out
`shared-discovery-bff`	Authenticated-to-IR adapter layer; parallel search; normalize to result cards; never expose AS session tokens to the browser
`as-search-adapter`	Maps unified query → AS Solr search params; maps hits → card schema; builds PUI URLs
`cs-search-adapter`	Maps unified query → gateway ES query; maps hits → card schema; builds Public Browser URLs
Config store	Environment / file / simple admin JSON for the single pair (URLs, credentials, labels, page size)

Technology choice (React/Vue/Svelte, Node/Java/Go) is **not** locked in v0.1; the contracts below are.

BFF HTTP surface (logical)

Method	Path	Purpose
`GET`	`/api/v1/health`	Liveness; optional upstream ping summary
`GET`	`/api/v1/search`	Federated search (see request/response)
`GET`	`/api/v1/config/public`	Non-secret UI config (labels, enabled sources, default page size)

No public write APIs. No proxy of raw AS/CS admin endpoints to the browser.

Search backends

ArchivesSpace

- Search is **Solr** behind routes documented under Search requests.
- Typical call (auth via `X-ArchivesSpace-Session` after `/users/:user/login`):

```
None
GET
/repositories/{repo_id}/search?q={lucene}&type[]=archival_object&type[]=resource&page=1&page_size=20
```

- Hits are **Solr documents**, not full JSONModels. Useful fields commonly include `title`, `primary_type` / `types`, `uri`, `identifier`, dates, and a string `json` blob for deeper mapping when needed.
- **Auth model for v1:** BFF holds a read-only username/password (or long-lived session refresh). Browser never sees the session header. Confirm with each deploy whether firewall policy allows only the BFF IP to reach `:8089`.
- **Publish/public visibility:** Prefer filters that match what the **PUI** would show (unpublished suppression). Exact filter_query flags are an open decision — do not return staff-only hits in a public UI.

CollectionSpace

- Public Browser talks to **Elasticsearch through `cspace-public-gateway`** (e.g. `https://{host}/gateway/{tenant}/es/...`), not directly to Nuxeo.
- The gateway already restricts to publicly published records (Publish To shortIds such as `all`, `cspacepub` — same set C1 reuses).
- BFF issues ES `_search` (or the same query shape the public browser uses) limited to **CollectionObject** (and media fields needed for thumbnails).
- **Auth model:** anonymous via gateway, same as the public browser.

What we deliberately do *not* query

- OAI-PMH endpoints (harvest protocol; not a search API)
- CS staff Common Services search (auth + over-broad visibility)
- Direct browser calls to AS `:8089`

Intermediate Result Card Schema

Normalize **after** search, at the BFF. Align display labels with the field maps below where practical.

Field	Type	Required	Notes
`id`	string	yes	Stable within source: AS `uri` or CS CSID / ES `_id`
`source`	`"archivesspace"` | `"collectionspace"`	yes	Drives labels / filters
`sourceUrl`	string	yes	Absolute link-out to AS PUI or CS Public Browser; may have to be generated from id and host config
`recordType`	string	yes	e.g. `archival_object`, `resource`, `collection_object`
`title`	string	yes	Display + link text
`identifier`	string	no	AS component id / CS objectNumber
`subjects`	string[]	no	Coarse filter; split on delimiter if search results are joined
`dateDisplay`	string	no	Pre-formatted; do not pretend full calendar normalization in v1; potentially extract year values for advanced filters
`description`	string	no	Truncate ~250 chars in UI
`creator`	string	no	
`thumbnailUrl`	string | null	no	Nice-to-have (Gap G-11)
`hasMedia`	boolean	no	Nice-to-have digitized signal
`foundIn`	string | null	no	AS hierarchy breadcrumb text; CS usually null (Gap G-10)

Display field matching: When rendering a card, use this schema's keys so the UI does not show both `Title` and `title` as separate rows. Source-specific labels (e.g. "Scope and Contents" vs "Brief Description") may still differ by `source` in Option A templates; Option B uses the unified labels above.

Field maps (AS / CS → result card)

The table below is a **target mapping from domain models**, not a guarantee of what live search responses contain.

Treat it as a starting hypothesis. Finalize card fields only after probing real ArchivesSpace Solr and CollectionSpace Elasticsearch (gateway) payloads — see Implementation notes.

Dublin Core analogue	System label (Proposed)	Display Label (Proposed)	ArchivesSpace Field	CollectionSpace Field
Identifier (link target)	`id` / `sourceUrl`	[Title]	Object:resource Property:uri "AppConfig[:public_proxy_url] + " uri	Public browser object URL (see C1 Gap G-13 for related public URL work)
source	`source`	Source	Object Type subset (archival_object, digital_object)	Record Type: Object
title	`title`	Title	Object:archival_object Property: Title	Title
identifier	`identifier`	Identifier	Object:archival_object Properties: component_id	objectNumber
subject	`subjects`	Category or Subject	Object:linked_agents Properties: IF role == subject	contentConcept
date	`dateDisplay`	Date Created	Object:dates	Pull earliest/latest scalar values from the date

Dublin Core analogue	System label (Proposed)	Display Label (Proposed)	ArchivesSpace Field	CollectionSpace Field
			Properties: IF ‘expression’, ELSE ‘begin + “...” + end’	details and concatenate them with '/'.
description	`description`	Description	Object:notes Properties: type = scopecontent	briefDescription
creator	`creator`	Creator	Object:linked_agents Properties: role == creator & NOT relator == ctb OR pbl	IF ‘objectProductionPerson’ ELSE ‘objectProductionOrganization’ ELSE ‘objectProductionPeople’
-	`thumbnailUrl`	N/A	Gap G-11 - Digital object preview	Gap G-11 - media thumbnail

Dublin Core analogue	System label (Proposed)	Display Label (Proposed)	ArchivesSpace Field	CollectionSpace Field
-	`hasMedia`	Includes digital media	Has `instance_do_link_rlshp` (digital object to instance) AND publish=true	Has (specific object record to media record relationship, G-15) AND PublishTo=true
-	`foundIn`	Found in	Hyperlinked breadcrumb of item's location in hierarchy; see Gap G-10	None

Implementation notes: search payloads vs domain models

This section is normative for planning: **do not freeze the intermediate schema, filters, or display templates until adapters have been exercised against reference endpoints with real data.**

1. **Card fields are bounded by search hits, not by full records.** Result cards are built from what each search endpoint returns (AS Solr documents; CS ES documents via the public gateway). Domain-model fields in the table above may exist on the JSONModel / Nuxeo record and still be absent, flattened, or renamed in the index. Optional card fields (`subjects`, `hasMedia`, `thumbnailUrl`, `foundIn`, etc.) ship only when the hit reliably supplies them.

2. **Spike before commit.** Before locking schema keys or UI filters, run representative searches against live (or staging) reference deployments for both sources. Capture sample hit documents, note which proposed card fields are present, empty, multi-valued,

or missing, and revise the table and adapters accordingly. Prefer documenting observed field names over continuing to guess from AS/CS UI forms.

3. **Index ≠ domain graph.** Relationships and structured sub-objects often do not survive indexing intact. Example: ArchivesSpace subjects are arrays of term records with properties such as term text and type (e.g. “Genre / Form”). Solr may expose a single joined `subjects` (or similar) string/array of labels and **drop type distinctions**, so genre vs topical subject cannot be filtered separately unless a typed field is confirmed in the index. The same class of loss can apply to creators, agents, dates, hierarchy (`foundIn`), and media links (`hasMedia` / thumbnails).

4. **Multivalued attributes.** When an endpoint returns multiple values for one logical field, the implementer may either:
    - keep the intermediate field as a list (`string[]`), or
    - join values with a delimiter for display

    Choose one convention per field and apply it consistently in both display options. Prefer lists when the UI needs multiselect filters.

5. **Pre-joined attributes.** When the index already concatenates several internal values (common for AS subjects), the implementer may split on a known delimiter to recover a list for filtering (e.g. multiselect). Only split when the delimiter is stable and values themselves do not contain it; otherwise treat the value as opaque display text.

6. **Dates and range filters.** Domain dates may be expressions, ranges, or partials. For year-only range filters in either display mode, extract clear four-digit years when possible and ignore or display-only ambiguous strings. Do not claim calendar-precise filtering unless the index exposes normalized date fields that both adapters can query the same way.

7. **Identifiers need endpoint confirmation.** Proposed mappings such as AS `component_id` are often blank on real records. The implementer must inspect hit payloads (and, if needed, the embedded `json` blob on AS Solr docs) to choose which field(s) populate `identifier` and `id` / `sourceUrl` for link-out. Prefer stable, usually-populated public identifiers over theoretically correct but empty model fields.

8. **Filters require queryable fields.** A facet or filter in the UI is only in scope if the upstream search API can constrain results by that field (or the BFF can derive it from fields present on every hit). Display-only enrichment that requires per-hit record GETs is out of scope for v1.

9. **Asymmetry is expected.** AS and CS will not expose parallel fields. It is acceptable for a card key to be populated for one source and null for the other; templates should omit empty rows rather than invent placeholder data.

10. **Revision rule.** After the reference-endpoint spike, update this field-map table to reflect **observed** index fields. Until then, treat rows as provisional and gaps (especially G-10, G-11, and media-related rows) as unresolved.

Display Implementation Options

Display Option A – Grouped / Side-by-side

- Two result groups: **Archival records** (AS) and **Museum objects** (CS).
- Each group has its **own** total count, page controls, and empty state.
- Source filters can hide a group entirely.
- Natural fit for federated search: no fake global ranking.
- In Lyrasis CST Interoperability Project user meetings, Display Option A was the preferred option from all (6) users interviewed

Search Results Display

Each feature is repeated for each data source in Display Option A.

Proposed Display Label	Proposed Field Behavior
View and Filter All [platform] results	Links to all results in ASpace PUI or CSpace PB
Previous | Showing results **# - #** | Next	Search navigation: Previous and next buttons link to sequential pages of search results
Page navigation	Current page of results highlighted with ‘previous’ and ‘next’ buttons to navigate through each platform’s search results

Record Metadata Display

Result card field	Proposed Display Label	AS Record Display	CS Record Display	Proposed Field Behavior
title	Title	x	x	Links to ASpace PUI or CSpace PB record

Result card field	Proposed Display Label	AS Record Display	CS Record Display	Proposed Field Behavior
recordType	Record Type	x	x	Not linked
identifier	Object Number		x	Not linked
identifier	Identifier	x		Not linked
dateDisplay	Date Made		x	Not linked
description	Brief Description		x	Display limited to 250 characters with link to read more
description	Scope and Contents	x		Display limited to 250 characters with link to read more
subjects	Category or Subjects	x	x	Linked, with sub-facets if possible (Gap G-14)
foundIn	Found in	x		Breadcrumbs link to the record in AS PUI
thumbnailUrl	none	x	x	Thumbnail / media preview

Display Option B – Unified List

- Single vertical list of cards, each stamped with source + record type.
- **v1 ranking rule:** do **not** interleave by foreign relevance scores. Prefer deterministic ordering, e.g. round-robin merge of the two current pages, or “AS block then CS block” within the viewport, with clear group headers still available.
- Source facet filters the list to one source (degrades to Option A with one group).

Search Results Display and Faceting

Proposed Display Label	Element type	Source field	Purpose
Previous | Showing results **#** **- #** | Next	Search navigation	N/A	Search navigation
Source system	Filter	`source` (see Configuration Requirements)	Filter results by CollectionSpace or ArchivesSpace
Creator	Free text box	`creator`	Refine by creator

Proposed Display Label	Element type	Source field	Purpose
Date - Earliest	Validated Text box (YYYY or YYYY-MM-DD)	`dateDisplay` / upstream date fields	Filter results by date
Date - Latest	Validated Text box (YYYY or YYYY-MM-DD)	`dateDisplay` / upstream date fields	Filter results by date
Subject	List with checkboxes	`linked_agents` / `contentConcept`	Filter results by category or subject (including genre/form if possible, Gap G-14)
Results Display	Controlled field	`hasMedia` / `thumbnailUrl`	Filter results by 'Records with Media' or 'All Records'

Record Metadata Display: Common Fields

See Intermediate Result Card Schema for detailed information on how to render ArchivesSpace and CollectionSpace fields.

Result card Field	Proposed Display Label	ArchivesSpace Field	CollectionSpace Field	Proposed Field Behavior
title	Title	Title	Title	Links to ASpace PUI or CSpace PB record
identifier	Identifier	component_id	objectNumber	Not linked
subjects	Category or Subject	linked_agents	contentConcept	Not linked
dateDisplay	Date Created	date	date	Not linked
description	Description	notes	briefDescription	Display limited to 250 characters with link to read more
thumbnailUrl	none	x	x	Thumbnail / media preview

Record Metadata Display: Source-specific Fields

Result card Field	Proposed Display Label	ArchivesSpace Field	CollectionSpace Field	Proposed Field Behavior
recordType	Type	Type subset (archival_object , resource)	CollectionObject	Not linked
foundIn	Found in	Hyperlinked breadcrumb of item's location in hierarchy	None	Breadcrumbs link to the record in AS PUI

Recommendation: Display Options

Recommendation: Ship **Option A as default**; implement Option B as an alternate layout flag once cards exist. Both share the same BFF response (`groups[]`).

BFF search contract (sketch)

Request

```
None
GET
/api/v1/search?q=pottery&sources=archivesspace,collectionspace&page
As=1&pageCs=1&pageSize=20&hasMedia=any
```

Param	Notes
`q`	Keyword; BFF maps to Lucene `q` (AS) and ES `query_string` / `multi_match` (CS)
`sources`	Subset of configured sources
`pageAs` / `pageCs`	Independent pagination (Option A). Option B UI may advance both or the active source
`pageSize`	Per source

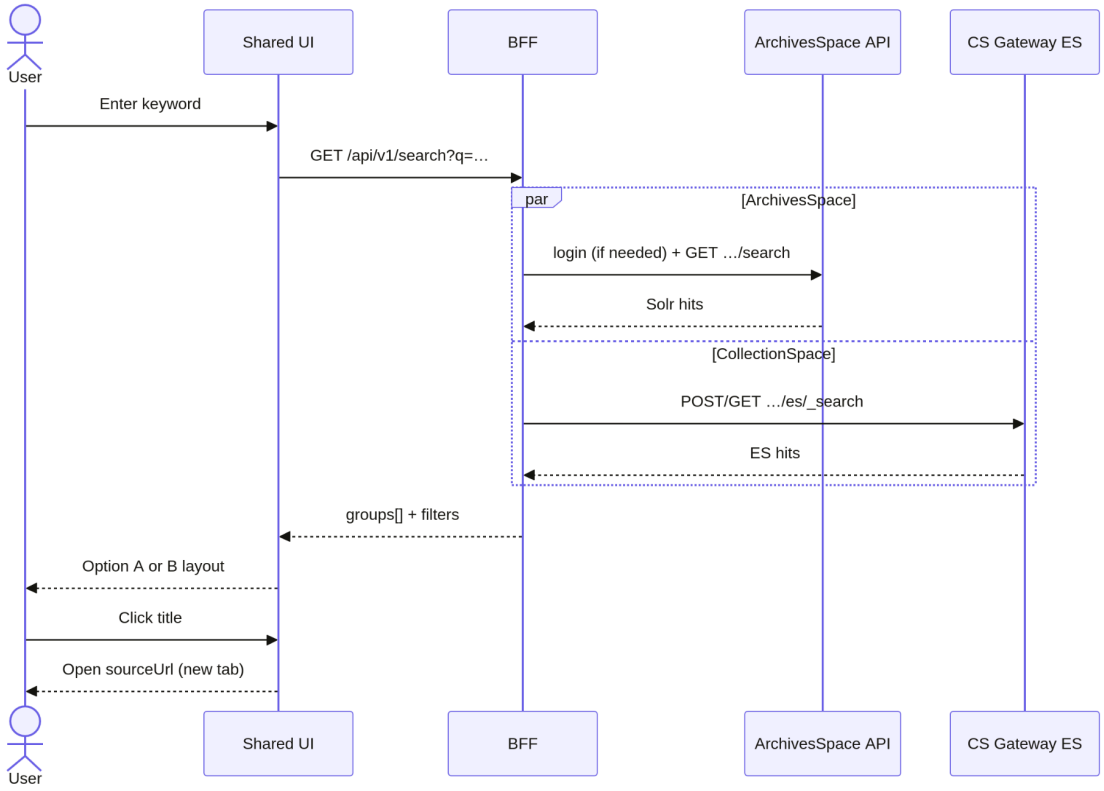
Param	Notes
`hasMedia`	`any` | `true` — best-effort; ignored if adapter cannot filter
`dateFrom` / `dateTo`	Optional coarse strings; adapter no-ops if unsupported

JSON

```
{
  "query": "pottery",
  "groups": [
    {
      "source": "archivesspace",
      "label": "Archival Records",
      "total": 42,
      "page": 1,
      "pageSize": 20,
      "status": "ok",
      "results": [ { "id": "/repositories/2/archival_objects/99", "title": "…",
"sourceUrl": "https://…" } ]
    },
    {
      "source": "collectionspace",
      "label": "Museum Objects",
      "total": 17,
      "page": 1,
      "pageSize": 20,
      "status": "ok",
      "results": [ { "id": "…", "title": "…", "sourceUrl": "https://…" } ]
    }
  ],
  "filters": {
    "sources": [
      { "value": "archivesspace", "count": 42 },
      { "value": "collectionspace", "count": 17 }
    ]
  }
}
```

If one upstream fails: that group returns `"status": "error"` (or `"degraded"`) with `results: []` and an opaque message; the other group still renders.

High-Level Sequence



API Dependencies

There are no known dependencies on the ArchivesSpace API. The ArchivesSpace functionality specified here is a Solr query, and API changes are not likely to affect the discovery product.

If the CollectionSpace Elasticsearch data model changes, this could cause functionality changes in the product. At the time of this writing (August 2026), the product team has discussed potential changes to the Elasticsearch data model, but these changes are not yet included in the CollectionSpace development roadmap. If the CollectionSpace ES model changes, then this product would need further development and review to ensure its continued functionality.

Behavior Scenarios

Note: Behavior Scenarios may be updated to align with Display Implementation Option, once a pathway is chosen.

BS01: End user searches and receives results from both ArchivesSpace and CollectionSpace

Step	Description
Given	The shared discovery app is configured for one ArchivesSpace instance and one CollectionSpace tenant.
	Both source systems are labeled and their public search APIs are reachable from the BFF.
When	An end user enters a keyword search in the unified discovery interface.
Then	Search results include records from both ArchivesSpace and CollectionSpace (subject to each backend's relevance ranking within its group).
	Each result card displays: title, source system label (e.g., 'Archival Records' / 'Museum Objects'), record type, date, brief description, and thumbnail (if available).
	Each source group shows its own result total (Option A); Option B may also show a combined count for display only.

BS02: End user filters results by source system

Step	Description
Given	Search results contain records from both ArchivesSpace and CollectionSpace.
When	The user selects 'Archival Records' in the source system facet.
Then	Only ArchivesSpace records are displayed.
	Selecting 'Museum Objects' shows only CollectionSpace records.

BS03: End user views a search result and navigates to the source system

Step	Description
Given	A search result record is displayed in the unified interface.
When	The user clicks the record title or a 'View full record' link.

Step	Description
Then	For an ArchivesSpace record: the user is taken to the corresponding record in the ArchivesSpace PUI.
	For a CollectionSpace record: the user is taken to the corresponding record in the CollectionSpace public browser.
	The link opens in a new tab.
	The user views a full metadata record in the source system, including fields beyond the shared preview card.

BS04: End user searches or facets by subject (unified interface only)

Step	Description
Given	A user has performed a successful search in a unified interface. *(OR a user is performing an advanced search. See Gap G-01.)*
When	The user views the result.
And	The user chooses from available subjects/categories to limit the results.
Then	The interface displays both ArchivesSpace and CollectionSpace records with records that contain the subject or category.

Error Scenarios

ES01: One or both source systems cannot provide data

Step	Description
Given	The shared discovery app is configured.
When	The user performs a search.
And	At least one upstream request fails because a source system is unable to provide data at this time.
Then	That group returns `"status": "error"` (or `"degraded"`) with `results: []` and an opaque message; any successful group still renders.

Other Requirements

Documentation

Documentation for this feature should include:

- How fields with no data are handled
- Required data
- Troubleshooting
- Governance, support, and contact information
- Accessibility features and compliance
- Reference documentation, such as field definitions

The documentation may include screenshots and examples.

Governance

The feature design should be reviewed, further edited, and if agreed upon, formally accepted by the CollectionSpace and ArchivesSpace product teams.

Maintenance of this product after the implementation phase will fall to the CollectionSpace and ArchivesSpace development team, managed by the Lyrasis Organizational Home Director of Community-Supported Technologies.

Error Reporting

The BFF logs each federated search (query parameters, which sources were called, per-source latency and outcome) and logs upstream endpoint responses (or a durable summary including status and error body when logging full payloads is impractical). Where the deployment supports it, operators may expose metrics such as per-endpoint failure rate, timeout rate, and latency percentiles on an admin dashboard. End-user messaging stays minimal: when a source fails, that source's result area is empty and the UI may show a short, opaque notice (for example a toast or inline alert) consistent with ES01 — no stack traces or internal URLs in the browser.

Performance and Scalability

The shared discovery UI and BFF are a small, purpose-built application and should be sized for expected concurrent users; a concrete traffic estimate is not yet available and should be revisited before production sizing. Each user's search fans out to both ArchivesSpace and CollectionSpace search endpoints, so shared-discovery load becomes additional read traffic on those systems. Both backends already serve search from dedicated indexes (Solr and

Elasticsearch), which are the appropriate tier to absorb that load; the BFF should use short timeouts and parallel calls so one slow upstream does not stall the whole page.

Implementation Gaps

See Resolved gaps between v0.3 and 0.5 for C3 spec for resolved specification gaps.

#	Gap	Description	Owner
G-11	Thumbnail	How will the system identify and render thumbnails or digital object previews? Is this nice to have or required?	Product teams, v1 Implementation team
G-12	CSpace Elasticsearch endpoint availability	Discuss with Lyrasis Hosting (Mark and Michael) about making the elasticsearch endpoint available outside of Lyrasis	Bridget, CSpac Product team
G-13	ASpace: Including digital objects in search	V1 implementation is specified to include hasMedia flag to limit search results to archival objects which have an associated DO. If many users request additional functionality for returning “orphaned” digital objects not associated with an AO, a v2 implementation could add DOs to search. DOs are included in solr index.	V2 Implementation Team
G-14	ASpace: Solr field names and results	What does the solr query return? Consultants couldn’t find this info in documentation. Required info for finishing the spec and describing subject faceting behavior. Does the solr query result for subject include subject types, e.g. genre/form?	ASpace Product Team, V1 Implementation Team
G-15	CSpace: Elasticsearch update for title	Construct an Elasticsearch "displaytitle" field from the underlying relevant CollectionSpace fields, that the specified feature can simply treat as a title. Does this action need to be repeated for all dc fields when they are mapped to CSpace arrays or multiple fields?	CSpace Product Team

#	Gap	Description	Owner
G-16	Governance	Review and consider more detail to the shared governance and maintenance plans during implementation	Bridget + both Product Teams, during implementation

Development Areas

#	Work item	Notes
D-A00	Reference-endpoint payload **spike** (AS Solr + CS gateway ES)	Capture sample hits; revise field maps before locking schema/filters. See Implementation notes
D-A01	Scaffold shared-discovery UI + BFF	Public read-only
D-A02	AS search adapter (AO + DO)	Session handling, PUI link builder, publish filters
D-A03	CS gateway ES adapter (CO)	Mirror public-browser eligibility
D-A04	Result card mapper + truncation	Unified keys; per-source label overlays for Option A
D-A05	Option A grouped UI + independent pagination	Default
D-A06	Option B unified layout flag	Same `groups[]` payload
D-A07	Coarse source filter + optional hasMedia	
D-A08	Partial-failure UX	One source down
D-A09	Deploy docs	Network allowlist for AS API; gateway URL; secrets